

14. Python

```
class ReporteManager:
    def generarReporteMensual(self, mes, año):
        # ANTES: versión vieja (30 líneas comentadas)
        """
        # def generarReporteMensual(self, mes, año):
        #     data = self.db.query(f"SELECT * FROM ventas WHERE mes={mes}")
        #     total = 0
        #     for venta in data:
        #         total += venta['monto']
        #     pdf = PDF()
        #     pdf.agregarTitulo(f"Ventas {mes}/{año}")
        #     pdf.agregarTabla(data)
        #     pdf.guardar(f"reporte_{mes}_{año}.pdf")
        #     return f"reporte_{mes}_{año}.pdf"
        """

        return self._generarNuevo(mes, año)

    def _generarNuevo(self, mes, año):
        pass # TODO: implementar

    def exportarExcel(self, data):
        """
        # EXCEL ANTIGUO - NO USAR
        # wb = xlwt.Workbook()
        # sheet = wb.add_sheet('Ventas')
        # for i, row in enumerate(data):
        #     for j, value in enumerate(row):
        #         sheet.write(i, j, value)
        # wb.save('ventas.xls')
        """

        print("Excel obsoleto")

    def backupAntiguo(self): pass
```

15. C++

```
class ValidadorGenerico {
private:
    std::map<std::string, std::function<bool(std::string)>> reglas;

public:
    ValidadorGenerico() {
        reglas["email"] = [](std::string s) { return s.find('@') != std::string::npos; };
        reglas["telefono"] = [](std::string s) { return s.length() == 10; };
        reglas["dni"] = [](std::string s) { return s.length() == 10 &&
std::all_of(s.begin(), s.end(), ::isdigit); };
        // 20+ reglas más...
    }

    bool validar(const std::string& campo, const std::string& valor) {
        auto it = reglas.find(campo);
        return it != reglas.end() ? it->second(valor) : false;
    }

    void agregarRegla(std::string nombre, std::function<bool(std::string)> func) {
        reglas[nombre] = func;
    }
};

class ValidadorSimple {
    bool esEmail(std::string email) { return email.find('@') != std::string::npos; }
};
```

16. PHP

```
class Producto {
    public $precio = 100;
}

function calcularTotal($productos) {
    $total = 0;
    foreach($productos as $p) {
        $total += $p->precio * 1.19; // ← CAMBIO AQUÍ
    }
    return $total;
}

function validarPrecio($precio) {
    return $precio > 0 && $precio < 100000; // ← CAMBIO AQUÍ
}

function generarReporte($productos) {
    foreach($productos as $p) {
        echo "Producto: {$p->nombre} - {$p->precio}"; // ← CAMBIO
    }
}

function notificarPrecio($producto) {
    if($producto->precio > 50) { // ← CAMBIO
        enviarEmail("Precio alto: {$producto->precio}");
    }
}

function actualizarStock($producto) {
    if($producto->precio < 10) restock(); // ← CAMBIO
}

function aplicarDescuento($precio) {
    return $precio > 80 ? $precio * 0.9 : $precio; // ← CAMBIO
}

function apiProductos() {
    return ['precio' => 100]; // ← CAMBIO
}
```


17. TypeScript

```
class Sistema {
  obtenerMontoFinal(usuarioId: string): number {
    const usuario = this.usuarios.get(usuarioId);
    const perfil = usuario?.getPerfil();
    const direccion = perfil?.getDireccionPrincipal();
    const ciudad = direccion?.getCiudad();
    const estado = ciudad?.getEstado();
    const pais = estado?.getPais();
    const zona = pais?.getZonaFiscal();
    const regimen = zona?.getRegimenImpositivo();
    const tasa = regimen?.getTasaIVA();
    const subtotal = this.calcularSubtotal();
    const total = subtotal * (1 + (tasa || 0.19));

    return total || 0;
  }

  getPerfil() { return this._perfil; }
  getDireccionPrincipal() { return this.direcciones[0]; }
  getCiudad() { return this._ciudad; }
  getEstado() { return this._estado; }
  getPais() { return this._pais; }
  getZonaFiscal() { return this._zona; }
  getRegimenImpositivo() { return this._regimen; }
  getTasaIVA() { return this.tasa; }
}
```

18. Python

```
def validar_email(email): return '@' in email
def validar_telefono(tel): return len(tel) == 10 and tel.isdigit()
def formatear_fecha(fecha): return fecha.strftime('%d/%m/%Y')
def calcular_iva(monto): return monto * 0.19
def generar_id(): return str(uuid.uuid4())
def enviar_email(to, subject, body): print(f"Email a {to}")
def log_error(error): print(f"ERROR: {error}")
def conectar_db(): return "DB conexión"
def sanitizar_string(s): return s.strip().lower()
def hash_password(password): return hashlib.md5(password.encode()).hexdigest()
# 70+ FUNCIONES MÁS...

class UsuarioService:
    def crear(self, data):
        from utils import validar_email, validar_telefono, log_error, conectar_db
        if not validar_email(data['email']): log_error("Email inválido")
        db = conectar_db()

class ProductoService:
    def actualizar(self, data):
        from utils import calcular_iva, enviar_email, generar_id
        total = calcular_iva(data['precio'])
        enviar_email("admin@", "Actualizado", generar_id())

class ReporteService:
    from utils import *
```